

01 Clickjacking

Clickjacking is when an attacker hides your site in an invisible frame on their page, then tricks someone into clicking something that looks harmless but actually triggers an action on your site underneath.

It's sneaky because nothing is broken — the user is really logged in and really clicking. They just don't know what they're actually clicking on.

The fix: headers like X-Frame-Options or Content-Security-Policy: frame-ancestors stop your site from being framed at all, shutting down the whole attack at once.

02 Command Injection

Command injection occurs when user input is passed on to a system shell without separating data from commands — a single semicolon or pipe can turn a form field into one that allows arbitrary code execution.

What is significant is not the complexity of the bug but the severity of its blast radius: whereas with SQL injection or XSS, the "interpreter" is the operating system itself, so that a successful exploit can compromise the entire machine, not just the application context.

07 Force Browsing

Forced browsing occurs when an attacker simply guesses the path — for example, /admin/reports — and then makes a direct request for that URL, without any need for a coding flaw.

It functions since "not linked" is equivalent to "not accessible," but merely hiding a link is not the same as protecting it; the only security it had was allowing the server to hand the page over to anyone who asked.

The fix: check permissions on every request, server-side. Do not just hide things from the UI.

10 Insecure URL Indirect

An insecure redirect occurs when an application directs users to a URL from user-controlled input (for example, redirect without checking that the URL points to a trusted destination—the attacker then creates a link that starts with your domain but leads to a phishing page).

What is subtle is that it exploits borrowed trust rather than code execution: the browser bar continues to display your domain right up to the redirect. The solution involves limiting the scope, not escaping characters—you should validate destinations against an allowlist, or better yet, use an internal key that maps to a fixed destination rather than accepting the full URL.

11 Leftover Debug Code

The debugging code that is left over is the kind that developers fail to remove before releasing a product—for example, a stray `console.log` that leaks tokens, a `hidden? debug=true` parameter which bypasses the login process, and a hardcoded test password included in a comment.

It's not a sophisticated exploit; it's simply a door that somebody forgot to shut. This is why it's dangerous — since it wasn't intended to be there, it doesn't usually appear in threat models, even though attackers still look for it.

The solution lies in the process, not in the code—scaffolding should be reviewed just as thoroughly as actual features, and debug flags must be removed automatically within the build pipeline rather than relying on people to remember.

14 Reflected XSS

Reflected XSS occurs when user input from a request—such as a search box or a URL parameter—is simply echoed back onto the page without being escaped, which means that a script tag entered into it is executed as actual code.

In contrast to stored XSS, nothing is stored on the server since the payload is contained within the request, typically in a specially constructed link which the user has to click on. This means that it is just as much a problem of social engineering as it is a coding issue.

The main takeaway is that the importance of the output is just as great as that of the input. The risky situation arises when data is written back into the page and the browser is unable to distinguish between attacker script and actual content. The solution involves escaping the output depending on the context (HTML, attribute, script) and also using a Content-Security-Policy as a safety net.

18 SQL Injection

SQL injection occurs when input given by a user is directly concatenated into a query rather than being handled as data – for example, the value ' OR '1'=1' ceases to be a search term and instead becomes part of the logic.

What should be noted is the extent to which this issue has already been resolved: parameterized queries almost entirely eliminate the vulnerability since the driver always deals with escaping correctly rather than relying on the person who wrote that line. Nevertheless, the vulnerability still exists — rarely because people are ignorant, more often due to legacy code or because of a single dynamic query that escapes review. Nowadays it is less a technical issue than one of discipline.

19 Stored Cross Site Scripting

21 Token Exposure in URL

Token exposure in URLs occurs when session tokens or reset links are passed as query parameters (for example, Token=abc123) rather than being included in the headers or in the request body.

The issue is that URLs end up appearing everywhere by default—in server logs, browser history, proxy caches, and the Rerefer header sent to any third-party resource the page loads. No mechanism "fails"; instead, the token simply passes through every stage where URLs are treated as harmless.

The solution is to avoid including secrets in the URL, make tokens expire after a short period, and render them invalid after first use.

22 User Enumeration

User enumeration occurs when an application's responses indicate which usernames or email addresses exist, even without a successful login — the fact that it gives a 'wrong password' message rather than a 'user not found' message is already a breach.

It is almost never the primary weakness; rather it relates to reconnaissance. When someone knows that a username is valid, brute-force or credential-stuffing attacks are reduced from having to guess two things to only having to guess one. In this instance, security ends up losing out to usability by default since detailed error messages seem more helpful, but that helpfulness constitutes the vulnerability. The solution (providing uniform responses and generic messages) involves making the product deliberately less clear to its users in order to become less clear to attackers.

23. Vertical Privilege Escalation

Vertical privilege escalation refers to a situation where a normal user acquires access that is intended for a more senior role—for example, the ability to carry out an admin function or view a dashboard for which they were not originally given the necessary permissions. This type of escalation involves moving up the hierarchy.

The typical reason is that privilege checks fail when someone is in the wrong location. If the admin button is hidden in the user interface, it achieves nothing, since the API will still carry out the action for anyone who calls it directly—the client can only suggest what is permitted, while the server enforces it.

The solution is to authorize each privileged action on the server side, for every request, according to the actual role—not based on what the user interface happened to display.

25 XML Injection

XML injection occurs when input given by a user is inserted into XML without the necessary escaping, which allows an attacker to include tags or attributes that the parser interprets as part of the structure rather than as data—this is the same kind of mixing of code and data that underlies SQL and command injection.

The special aspect relating to XML is the use of external entities: if malicious markup is inserted it can cause XXE, allowing an attacker to retrieve local files or make outgoing requests and thus turn a parsing vulnerability into either file disclosure or SSRF. The solution is to use proper escaping or serialization libraries rather than string concatenation, together with disabling external entity resolution by default – this being a safe default setting which many parsers only made standard after many years of leaving it as an optional feature.